| Register No. | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

**SRM Institute of Science and Technology**
**College of Engineering and Technology**
**School of Computing**
**(Common to all branches)**

**SET - A**

**Academic Year: 2023-24 (ODD)**

**Test: CLA-T2** **Date: -11-2023**
**Course Code & Title: 21CSC203P Advanced Programming Practice** **Duration:** 100 minutes
**Year & Sem: II Year / III Sem** **Max. Marks:** 50

**Course Articulation Matrix:** ***(to be placed)***

| S.No. | Course Outcome | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 | PSO3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | **CO1** | 3 | 2 | - | - | - | - | - | - | - | - | - | - | - | 2 | - |
| 2 | **CO2** | 3 | 2 | - | 1 | - | - | - | - | - | - | - | - | - | 2 | - |
| 3 | **CO3** | 3 | - | 2 | - | 2 | - | - | - | - | 1 | - | - | - | 2 | - |
| 4 | **CO4** | 3 | 2 | - | 1 | - | - | - | - | - | - | - | - | - | 2 | - |
| 5 | **CO5** | 3 | - | 2 | 1 | 2 | - | - | - | - | 1 | - | - | - | 2 | - |

**Part – A**
**(5 x 4 = 20 Marks)**
**Answer any 4 Questions**

| | | | | | | |
|---|---|---|---|---|---|---|
| **1** | Write a program to create an applet for displaying a rectangle with the string "Rectangle Shape".<br>Ans:<br>import java.applet.*;<br>import java.awt.*;<br>public class rect_applet extends Applet<br>{<br>int height, width;<br>public void init()<br>{<br>height = getSize().height;<br>width = getSize().width;<br>setName("rect_Applet");<br>}<br>public void paint(Graphics g)<br>{<br>g.drawRoundRect(10, 30, 120, 120, 2, 3);<br>g.drawString("Rectangle Shape ",10,50);<br>}<br>} | **4** | **L3** | **3** | **3** | **1.4.1** |
| **2** | What is JDBC Driver and explain their types?<br>Ans:<br>JDBC Driver is a software component that enables Java application to interact with the database. There are 4 types of JDBC drivers:<br>1. **JDBC-ODBC bridge driver:** The JDBC-ODBC | **4** | **L2** | **3** | **1** | **1.4.1** |

| | | | | | | |
|---|---|---|---|---|---|---|
| | bridge driver uses the ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of the thin driver. It is easy to use and can be easily connected to any database.<br>2. **Native-API driver (partially java driver):** The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in Java. Its performance is better than JDBC-ODBC bridge driver. However, the native driver must be installed on each client machine.<br>3. **Network Protocol driver (fully java driver):** The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is entirely written in Java. There is no requirement of the client-side library because of the application server that can perform many tasks like auditing, load balancing, logging, etc.<br>4. **Thin driver (fully java driver):** The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as the thin driver. It is entirely written in Java language. Its performance is better than all other drivers however these drivers depend upon the database. | | | | | |
| **3** | What are the advantages of using MVC in developing complex applications?<br>Ans:<br>1. Modularization and Separation of Concerns: MVC enforces a clear separation between the Model, View, and Controller components. This separation enhances code organization and maintainability by isolating different aspects of the applications functionality.<br>2. Scalability: In complex applications, adding new features or modifying existing ones can be challenging. MVC's modular structure allows developers to extend or modify one component without affecting others, making the application more scalable.<br>3. Parallel Development: Different teams can work on different components concurrently. This parallel development accelerates the overall development process.<br>4. Reusability: Components in MVC can often be reused across different parts of the application or even in other projects, reducing redundancy and development time.<br>5. Maintenance and Updates: MVC simplifies maintenance as changes in one component do not necessarily require changes in others. This reduces the risk of introducing unintended side effects.<br>6. User Interface Adaptability: The View in MVC handles the user interface, making it easier to create different interfaces for various devices or platforms while keeping the underlying logic consistent. | **4** | **L3** | **3** | **1** | **1.4.1** |
| **4** | i. What is functional programming? List the Characteristics of functional programming.<br>Ans:<br>Functional programming is a programming paradigm in which it is tried to bind each and everything in pure mathematical functions. It is a declarative type of programming style that | **4** | **L1** | **4** | **1** | **1.4.1** |

| | | | | | | |
|---|---|---|---|---|---|---|
| | focuses on what to solve rather than how to solve.<br>**Characteristics**<br>• Functional programming method focuses on results, not the process<br>• Emphasis is on what is to be computed<br>• Data is immutable<br>• Functional programming decompose the problem into functions<br>• It is built on the concept of mathematical functions which uses conditional expressions and recursion to do perform the calculation<br>• Functional programming languages are designed on the concept of mathematical functions that use conditional expressions and recursion to perform computation.<br>• Functional programming supports higher-order functions and lazy evaluation features.<br>• Functional programming languages don't support flow Controls like loop statements and conditional statements like If-Else and Switch Statements. They directly use the functions and functional calls.<br><br>ii. What is the output of the following code segment?<br>c = map(lambda x:x+x,filter(lambda x: (x>=3), (1,2,3,4)))<br>print(list(c))<br>Ans:<br>6,8 | | | | | |
| **5** | Given that fact that: Lucy is a Professor, Danny is a Professor, James is a Lecturer, All professors are Dean<br>Write a Query to reterive all deans?<br>Ans:<br>from pyDatalog import pyDatalog<br>pyDatalog.create_terms('X,professor,lecturer, dean')<br>+professor('lucy')<br>+professor('danny')<br>+lecturer('james')<br>dean(X)<=professor(X)<br>print(dean(X)) | **4** | **L5** | **4** | **2** | **1.4.1** |
| **6** | Write a Python SymPy program to solve the following<br>i. sym.diff(sym.sin(x), x)<br>cos(x)<br>ii. Evaluate Pi<br>sym.pi.evaluf()<br>Find the output of the following<br>iii. sym.integrate(2*x+sym.sinh(x),x)<br>x**2+cosh(x)<br>iv. sym.expand(sym.cos(x+y),trig=true)<br>-sin(x)sin(y)+cos(x)cos(y) | **4** | **L3** | **5** | **3** | **1.4.1** |
| **7** | Explain about the options available with grid() method of Geometry manager.<br>Ans:<br>Column : This option specifies the column number in which the widget is to be placed. The index of leftmost column is 0.<br>Row: This option specifies the row number in which the widget is to be placed. The topmost row is represented by 0.<br>Columnspan : This option specifies the width of the widget. It mainly represents the number of columns up to which, the column is expanded. | **4** | **L2** | **5** | **1** | **1.4.1** |

| | | | | | | |
|---|---|---|---|---|---|---|
| | Rowspan: This option specifies the height of the widget. It mainly represents the number of rows up to which, the row is expanded.<br>padx, pady: This option mainly represents the number of pixels of padding to be added to the widget just outside the widget's border.<br>ipadx, ipady: This option is mainly used to represents the number of pixels of padding to be added to the widget inside the widget's border.<br>Sticky: If any cell is larger than a widget, then sticky is mainly used to specify the position of the widget inside the cell. It is basically concatenation of the sticky letters which represents the position of the widget. It may be N, E, W, S, NE, NW, NS, EW, ES. | | | | | |
| | **Part – B**<br>**(3 x 10 = 30 Marks)** | | | | | |
| **8 a** | i. What is a thread? How will you set priority to a thread?<br>Ans:<br>A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a *thread,* and each thread defines a separate path of execution.<br>Thread priorities are used by the thread scheduler to decide when each thread should be allowed to run. To set a thread's priority, use the **setPriority( )** method, which is a member of **Thread**. This is its general form:<br>final void setPriority(int *level*)<br>Here, *level* specifies the new priority setting for the calling thread. The value of *level* must be within the range **MIN_PRIORITY** and **MAX_PRIORITY**. Currently, these values are 1 and 10, respectively. To return a thread to default priority, specify **NORM_PRIORITY**, which is currently 5. These priorities are defined as **final** variables within **Thread**. We can obtain the current priority setting by calling the **getPriority( )** method of **Thread**, shown here:<br>final int getPriority( ) | **2** | **L2** | **3** | **1** | **1.4.1** |
| | ii. Develop a Java program to process the mathematical equation: p = sin (x) + cos (y) + tan (z) using multithreading concept. Since the trigonometric functions are independent operations without any dependencies between them, they can be executed concurrently. After that their results can be combined to produce the final result. The main/default thread is called MathThreads, which acts like a master thread. It creates three worker threads (MathSin, MathCos, and MathTan) and assigns them to compute values for different data inputs. All three worker threads are concurrently executed on shared or dedicated CPUs depending on the type of machine. Although the master thread can continue its execution, in this case, it needs to make sure that all operations are completed before combining individual results. This is accomplished by waiting for each thread to complete by invoking join() method associated with each worker thread.<br>Ans:<br>import static java.lang.Math.*;<br>class A extends Thread | **8** | **L5** | **3** | **3** | **1.4.2** |

| | | | | | | |
|---|---|---|---|---|---|---|
| | {<br>double res1;<br>public void run ()<br>{<br>res1 = sin(Math.toRadians(90)) ;<br>System.out.println(res1);<br>}<br>}<br>class B extends Thread<br>{<br>double res2;<br>public void run ()<br>{<br>res2 = cos(Math.toRadians(0)) ;<br>System.out.println(res2);<br>}<br>}<br>class C extends Thread<br>{<br>double res3;<br>public void run ()<br>{<br>res3 = tan(Math.toRadians(45)) ;<br><br>System.out.println(res3);<br>}<br>}<br>class trigo<br>{<br>public static void main (String args[])<br>{<br>A threadA = new A();<br>B threadB = new B();<br>C threadC = new C();<br>System.out.println(“Start Thread A”);<br>threadA.start();<br>System.out.println(“Start Thread B”);<br>threadB.start();<br>System.out.println(“Start Thread C”);<br>threadC.start();<br>try<br>{<br>threadA.join();<br>threadB.join();<br>threadC.join();<br>}<br>catch (InterruptedException e)<br>{<br>System.out.println("Interrupted");<br>}<br>System.out.println(threadA.res1+threadB.res2+threadC.res3);<br>}<br>} | | | | | |
| | **Or** | | | | | |
| **8 b** | List the steps involved in establishing a JDBC connection with MySQL database and explain each step with code snippets.<br>Ans:<br>The following are the steps for connectivity between Java Program and Database: | **10** | **L1** | **3** | **1** | **1.4.1** |

1. Import the Packages
2. Load the drivers using the *forName() method*
3. Register the drivers *using DriverManager*
4. Establish a connection *using the Connection class object*
5. Create a statement
6. Execute the query
7. Close the connections

**Step 1:** Import the Packages

**Step 2:** Loading the drivers
In order to begin with, you first need to load the driver or register it before using it in the program. Registration is to be done once in your program. You can register a driver in one of two ways mentioned below as follows:

**2-A** Class.forName()
Here we load the driver's class file into memory at the runtime. No need of using new or create objects. The following example uses Class.forName() to load the Oracle driver as shown below as follows:

Class.forName("oracle.jdbc.driver.OracleDriver");

**2-B** DriverManager.registerDriver()
DriverManager is a Java inbuilt class with a static member register. Here we call the constructor of the driver class at compile time. The following example uses DriverManager.registerDriver()to register the Oracle driver as shown below:

DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver())

**Step 3:** Establish a connection *using* the *Connection class object*
After loading the driver, establish connections as shown below as follows:

Connection con = DriverManager.getConnection(url,user,password)

- **user: U**sername from which your SQL command prompt can be accessed.
- **password:** password from which the SQL command prompt can be accessed.
- **con:** It is a reference to the Connection interface.
- **Url**: Uniform Resource Locator which is created as shown below:

String url = " jdbc:oracle:thin:@localhost:1521:xe"

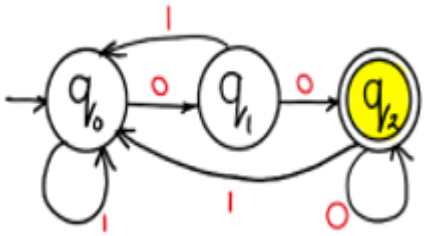
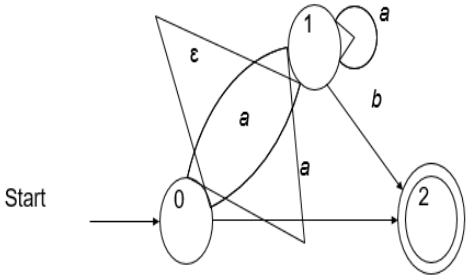
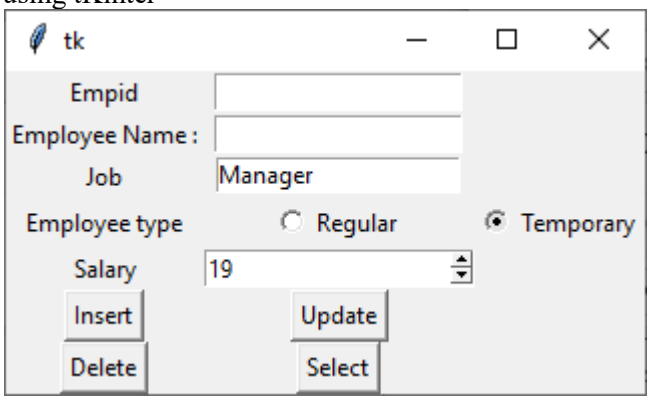
Where oracle is the database used, thin is the driver used, @localhost is the IP Address where a database is stored, 1521 is the port number and xe is the service provider. All 3 parameters above are of String type and are to be declared by the programmer before calling the function. Use of this can be referred to form the final code.

**Step 4:** Create a statement
Once a connection is established you can interact with the database. The JDBCStatement, CallableStatement, and

| | | | | | | |
|---|---|---|---|---|---|---|
| | PreparedStatement interfaces define the methods that enable you to send SQL commands and receive data from your database.<br>Use of JDBC Statement is as follows:<br>Statement st = con.createStatement();<br>***Note:*** *Here, con is a reference to Connection interface used in previous step .*<br>**Step 5:** Execute the query<br>Now comes the most important part i.e executing the query. The query here is an SQL Query. Now we know we can have multiple types of queries. Some of them are as follows:<br>• The query for updating/inserting a table in a database.<br>• The query for retrieving data.<br>The executeQuery() method of the **Statement interface** is used to execute queries of retrieving values from the database. This method returns the object of ResultSet that can be used to get all the records of a table.<br>The executeUpdate(sql query) method of the Statement interface is used to execute queries of updating/inserting.<br>**Pseudo Code:**<br>int m = st.executeUpdate(sql);<br>if (m==1)<br>System.out.println("inserted successfully : "+sql);<br>else<br>System.out.println("insertion failed"); | | | | | |
| **9 a** | i. Explain Concurrent Programming and its issues.<br>Ans:<br>Concurrent programming is programming with multiple tasks. The major issues of concurrent programming are:<br>1. Sharing computational resources between the tasks;<br>2. Interaction of the tasks. | **3** | **L2** | **4** | **1** | **2.2.4** |
| | i. Explain Concurrent Programming and its issues. Write a program to implement Race Condition in Python.<br>Ans:<br>Issues:<br>Race Condition, Deadlock, Resource Starvation, Priority Inversion.<br>Race Condition<br>from threading import Thread<br>x = 0<br>def add_one():<br>global x<br>for i in range(1000):<br>x += 1<br>def subtract_one(): | **7** | **L5** | **4** | **1** | **2.1.3** |

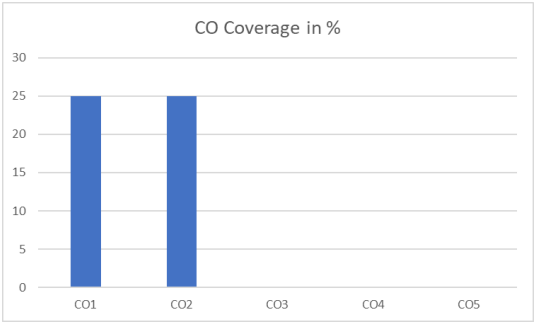
| | | | | | | |
|---|---|---|---|---|---|---|
| | global x<br>for i in range(1000):<br>x -= 1<br><br>thread1 = Thread(target=add_one)<br>thread2 = Thread(target=subtract_one)<br><br>thread1.start()<br>thread2.start()<br>print(x) | | | | | |
| | **Or** | | | | | |
| **9 b** | i. Explain briefly about any 4 socket methods.<br>Ans:<br>sock_object.recv():<br>Use this method to receive messages at endpoints when the value of the protocol parameter is TCP.<br>sock_object.send():<br>Apply this method to send messages from endpoints in case the protocol is TCP.<br>sock_object.recvfrom():<br>Call this method to receive messages at endpoints if the protocol used is UDP.<br>sock_object.sendto():<br>Invoke this method to send messages from endpoints if the protocol parameter is UDP.<br>sock_object.gethostname():<br>This method returns hostname.<br>sock_object.close():<br>This method is used to close the socket. The remote endpoint will not receive data from this side. | **4** | **L2** | **4** | **1** | **1.4.1** |
| | ii. Write a Socket based Client Server Application to simulate a Simple Chat.<br>Ans:<br>#SERVER CODE<br>import socket<br>import threading<br>def handle_client(client_socket, client_address):<br># receive the first message from the client<br>client_message = client_socket.recv(1024).decode('utf-8')<br>print('Received message from {}: {}'.format(client_address, client_message))<br>while client_message != 'quit':<br># send the message to all other clients<br>for client in clients:<br>if client != client_socket:<br>client.sendall(client_message.encode('utf-8'))<br># receive the next message from the client client_message = client_socket.recv(1024).decode('utf-8')<br>print('Received message from {}: {}'.format(client_address, client_message))<br># remove the client from the list of clients<br>clients.remove(client_socket)<br># close the connection<br>client_socket.close()<br>def main():<br># create a socket object | **6** | **L3** | **4** | **2** | **1.5.2** |

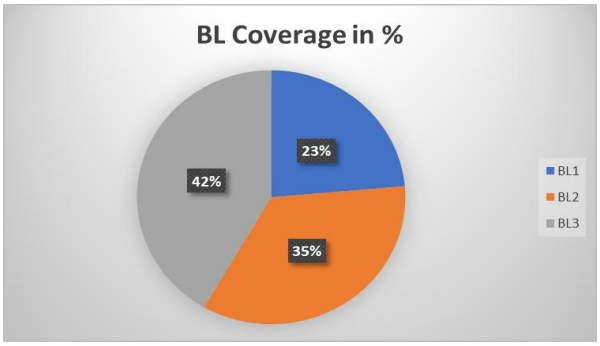
| | | | | | | |
|---|---|---|---|---|---|---|
| | server_socket = socket.socket(socket.AF_INET,<br>socket.SOCK_STREAM)<br># get local machine name<br>host = socket.gethostname()<br># bind the socket to a public host and a well-known port<br>server_socket.bind((host, 9999))<br># listen for incoming connections<br>server_socket.listen(5)<br>print('Server listening on {}:{}'.format(host, 9999))<br>while True:<br># wait for a new client to connect<br>client_socket, client_address = server_socket.accept()<br>print('Accepted connection from<br>{}:{}'.format(client_address[0], client_address[1]))<br># add the client to the list of clients<br>clients.append(client_socket)<br># start a new thread to handle the client<br>client_thread = threading.Thread(target=handle_client,<br>args=(client_socket, client_address))<br>client_thread.start()<br>if name == ' main ':<br># list of connected clients<br>clients = []<br>main()<br>#CLIENT CODE<br>import socket<br>import threading<br>def receive_messages():<br>while True:<br># receive messages from the server and print them to the<br>console<br>data = client_socket.recv(1024).decode('utf-8')<br>print(data)<br>def main():<br># create a socket object<br>client_socket = socket.socket(socket.AF_INET,<br>socket.SOCK_STREAM)<br># get local machine name<br>host = socket.gethostname()<br># connect to the server<br>client_socket.connect((host, 9999))<br># start a new thread to receive messages from the server<br>receive_thread = threading.Thread(target=receive_messages)<br>receive_thread.start()<br>while True:<br># read a message from the console and send it to the server<br>message = input()<br>client_socket.sendall(message.encode('utf-8'))<br># close the connection<br>client_socket.close()<br>if name == ' main ':<br>main() | | | | | |
| **10 a** | i. Draw the DFA for the language accepting strings ending with '00' over input alphabets ∑={0, 1}. | **5** | **L2** | **5** | **1** | **1.3.2** |

| | | | | | | |
|---|---|---|---|---|---|---|
| |  <br>ii. Construct the NFA and give the transition table for the regular expression a+aa*b+a*b.<br>  | **5** | **L2** | **5** | **1** | **2.3.2** |
| | **Or** | | | | | |
| **10 b** | i. What is Event Handler in Event Driven Programming?<br>Ans:<br>In programming, an event handler is a callback routine that operates asynchronously once an event takes place. It dictates the action that follows the event. The programmer writes a code for this action to take place. An event is an action that takes place when a user interacts with a program.<br>ii. Write a python code to Design the following GUI screen using tKinter<br>  | **3**<br><br>**7** | **L1**<br><br>**L5** | **5**<br><br>**5** | **1**<br><br>**3** | **2.2.4**<br><br>**2.4.3** |

***Program Indicators are available separately for Computer Science and Engineering in AICTE examination reforms policy.**

**Course Outcome (CO) and Bloom's level (BL) Coverage in Questions**



**Approved by the Audit Professor/Course Coordinator**